

Supplement to
*Generators and Relations for
Discrete Groups*

Hilbert Reconstruction and Lean Formalization

Jurek Róžański

Military University of Technology, Warsaw, Poland

CGJTEAMLAB

August 27, 2026

Document Status

This volume is a research supplement accompanying the CGJteamLab formalization project. Its purpose is to investigate the generators-and-relations viewpoint of Coxeter and Moser inside a synthetic Hilbert geometry formalized in Lean.

The source book is H. S. M. Coxeter and W. O. J. Moser, *Generators and Relations for Discrete Groups*, second edition.

The present supplement does not reproduce the source book. Instead, it develops a parallel formal program:

Hilbert geometry $\longrightarrow$ geometric transformations $\longrightarrow$ generators $\longrightarrow$ words $\longrightarrow$ relations.

The Lean source is authoritative for theorem statements, assumptions, dependencies, and proof status. This document records the mathematical architecture, formal dependency structure, and relation to the Coxeter–Moser viewpoint.

At the present checkpoint the first three stages of the supplement are implemented: a geometric line reflection has been reconstructed, words in two reflection generators have been defined, and the first nontrivial pairwise relation has been proved for perpendicular axes:

$$(r_1 r_2)^2 = 1.$$

Contents

Document Status	iii
Research Program	vii
1 Line Reflection as a Hilbert Generator	1
1.1 Purpose and Status	1
1.2 Coxeter Motivation	1
1.3 Geometric Design	2
1.3.1 Why the construction begins as a relation	2
1.3.2 ReflectionAxis	2
1.3.3 PerpendicularToAxis	2
1.3.4 IsLineReflection	3
1.4 Existence of the Reflected Point	3
1.5 Relational Involution	4
1.6 Fixed Points	4
1.7 Right-Angle Carrier Transport	5
1.8 Uniqueness of the Perpendicular Foot	5
1.9 Uniqueness of Reflection	6
1.10 The Reflection Operator	6
1.11 The First Coxeter Relation	7
1.12 Permutation Packaging	7
1.13 Dependency Summary	8
1.14 Axiomatic Status	8
1.15 Architectural Significance	8
1.16 Next Checkpoint	9
1.17 Conclusion	9
2 Words in Reflection Generators	11
2.1 Purpose of the Module	11
2.2 Product of Two Reflection Generators	11
2.3 The Coincident-Axis Check	12
2.4 Powers of a Reflection Product	12
2.5 The Pairwise Coxeter Relation	13
2.6 Why the Word Layer is Separate	13
2.7 Axiomatic Status	14
2.8 Conclusion	14
3 Relations Between Two Reflection Generators	15
3.1 Purpose of the Chapter	15
3.2 Reflection as a Synthetic Isometry	15
3.3 Perpendicular Reflection Axes	16
3.4 Setwise Preservation of the Perpendicular Axis	17

3.5	Transport of Midpoints and Perpendiculars	17
3.6	Conjugation	18
3.7	Commutation	19
3.8	The Square of the Product	19
3.9	The Coxeter Relation $p_{12} = 2$	20
3.10	Dependency Architecture	20
3.11	Axiomatic Status	21
3.12	Conclusion	21
A	Source Map: Coxeter–Moser	23
A.1	Section 1.1: Generators and Relations	23
A.2	Section 4.1: Cyclic and Dihedral Groups	23
A.3	Section 4.3: Groups Generated by Reflections	23
A.4	Sections 9.1–9.3: Groups Generated by Reflections	23
B	Lean Module Map	25
	Checkpoint	27

Research Program

The initial research program is deliberately narrow.

1. Reconstruct reflection in a line from the established Hilbert incidence, order, congruence, midpoint, and right-angle theory.
2. Promote the geometric reflection relation to an actual permutation of the point set.
3. Prove the first Coxeter relation

$$r_l^2 = 1.$$

4. Construct words in several reflection generators.
5. Study the pairwise relations

$$(r_l r_m)^p = 1$$

for intersecting axes.

6. Treat the parallel-axis case, corresponding to infinite dihedral behavior and translation.
7. Use selected propositions of Euclid's *Elements* as benchmark configurations for the generators-and-relations language.

Items 1–4 are complete. Item 5 has now been entered through its first nontrivial instance: perpendicular axes give the Coxeter exponent $p = 2$.

The principal Coxeter–Moser source locations for the opening phase are Sections 1.1, 4.1, 4.3, and 9.1–9.3.

Chapter 1

Line Reflection as a Hilbert Generator

1.1 Purpose and Status

This document records the first completed component of the experimental Coxeter layer of CGJteamLab. The source module is

```
CGJteamLab/Coxeter/Reflection.lean
```

The module is deliberately separated from the flat sequence of Euclid Proposition files. It is not another proposition of the *Elements*; it is a transformation layer derived over the same Hilbert foundation.

The immediate objective is to obtain a genuine reflection generator

$$r_l : \text{Point} \longrightarrow \text{Point}$$

without introducing reflection as a new primitive geometric operation. The construction starts with a geometric relation, proves existence and uniqueness, and only then packages the relation as a point transformation.

The current file compiles cleanly. No new geometric axiom and no `sorry` is introduced locally.

1.2 Coxeter Motivation

Coxeter and Moser begin with generators and defining relations and later specialize to groups generated by reflections. In the reflection-group language, each basic reflection is involutory:

$$r_i^2 = 1,$$

while the geometry of pairs of reflecting hyperplanes is encoded by the periods of products

$$(r_i r_j)^{p_{ij}} = 1.$$

Their graphical notation records these pairwise periods by a Coxeter diagram.

The present module addresses only the first relation. It constructs a single Hilbert line reflection and proves, in literal operator form,

$$r_l(r_l(P)) = P.$$

Products of distinct reflections and relations of the form $(r_l r_m)^p = 1$ belong to the next layer and are intentionally absent from this file.

The relevant source locations in Coxeter–Moser, *Generators and Relations for Discrete Groups*, second edition, are Sections 1.1, 4.3, and 9.1–9.3.

1.3 Geometric Design

1.3.1 Why the construction begins as a relation

The Hilbert development does not take a global transformation space as a primitive object. The natural first notion is therefore a relation between a point P and its reflected point P' .

For a point off the axis l , reflection is characterized by a foot H satisfying

$$H \in l, \quad P - H - P', \quad PH \cong HP',$$

together with the condition that PH is perpendicular to the axis.

For a point on the axis, the reflected point is the point itself.

Thus the geometric core is

$$P' = P \text{ on } l \quad \text{or} \quad H \in l, H = \text{mid}(P, P'), PH \perp l.$$

1.3.2 ReflectionAxis

The current Hilbert API often represents a line geometrically by two distinct points known to lie on it. The reflection axis therefore stores both the carrier and witnesses sufficient for perpendicular constructions.

```
structure ReflectionAxis
  [HilbertIncidence Geo] where
  carrier : Geo.Line
  A : Geo.Point
  B : Geo.Point
  hAB : Not (A = B)
  hA : HilbertIncidence.OnLine A carrier
  hB : HilbertIncidence.OnLine B carrier
```

This is not additional geometry. It packages the nondegenerate line data already required by the existing Hilbert construction interface.

1.3.3 PerpendicularToAxis

Perpendicularity to the axis is represented using the established point-based right-angle language.

```

def PerpendicularToAxis
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis : ReflectionAxis Geo)
  (H P : Geo.Point) : Prop :=
And
  (HilbertIncidence.OnLine H axis.carrier)
  (And
    (Not (HilbertIncidence.OnLine P axis.carrier))
    (Exists fun R : Geo.Point =>
      And
        (HilbertIncidence.OnLine R axis.carrier)
        (And
          (Not (R = H))
          (HilbertRightAngle Geo R H P))))))

```

The witness R supplies a nondegenerate direction of the carrier. No primitive relation “two lines are perpendicular” is added.

1.3.4 IsLineReflection

The reflection relation has a fixed-point branch and an off-axis branch.

```

def IsLineReflection
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis : ReflectionAxis Geo)
  (P P' : Geo.Point) : Prop :=
Or
  (And
    (HilbertIncidence.OnLine P axis.carrier)
    (P' = P))
  (And
    (Not (HilbertIncidence.OnLine P axis.carrier))
    (Exists fun H : Geo.Point =>
      And
        (PerpendicularToAxis Geo axis H P)
        (HilbertIsMidpoint Geo H P P'))))

```

The midpoint condition packages both strict betweenness $P - H - P'$ and equality of the two half-segments.

1.4 Existence of the Reflected Point

The first main theorem is

```

theorem line_reflection_exists
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis : ReflectionAxis Geo)
  (P : Geo.Point) :

```

```
exists P' : Geo.Point,
  IsLineReflection Geo axis P P'
```

The proof splits on whether P lies on the axis.

If $P \in l$, choose $P' = P$.

If $P \notin l$, Euclid I.12 in its Hilbert reconstruction supplies the perpendicular foot H . The established theorem

```
hilbert_extend_segment_beyond
```

then extends PH beyond H to a point P' satisfying

$$P - H - P', \quad PH \cong HP'.$$

This is exactly the midpoint condition needed by `HilbertIsMidpoint`.

The conceptual dependency is therefore

perpendicular construction + equal segment extension $\implies$ existence of reflection.

1.5 Relational Involution

Before constructing a function, the development proves that the reflection relation is symmetric:

```
theorem line_reflection_symmetric
...
(hRef : IsLineReflection Geo axis P P') :
  IsLineReflection Geo axis P' P
```

For off-axis points, the same midpoint H is used in the reverse direction. The substantive geometric obligation is to transport the right-angle condition from the ray HP to the opposite ray HP' . This uses the already developed theory of right-angle transport and opposite extensions.

This theorem is the relational precursor of the Coxeter involution relation:

$$P R_l P' \implies P' R_l P.$$

1.6 Fixed Points

The module records the exact fixed-point set of the reflection:

```
theorem line_reflection_fixed_iff_on_axis
...
(P : Geo.Point) :
  IsLineReflection Geo axis P P <->
    HilbertIncidence.OnLine P axis.carrier
```

Thus

$$r_l(P) = P \iff P \in l$$

once the functional reflection has been constructed.

The companion theorem `line_reflection_moves_off_axis` states that an off-axis point cannot be fixed.

1.7 Right-Angle Carrier Transport

The uniqueness proof exposed a reusable gap in the permanent Hilbert API: a right angle should not depend on which non-vertex point is chosen on the same carrier of one arm.

Three local theorems were therefore established:

```
coxeter_right_angle_sameRay_first
coxeter_right_angle_opposite_first
coxeter_right_angle_collinear_first
```

The first transports a right angle along the same ray. The second reverses the first arm across the vertex. The third combines the two cases using betweenness trichotomy and gives the carrier-level statement required for perpendicular uniqueness.

This is a useful example of the formal difference between an intuitive statement about perpendicular lines and the point/ray representation used in the Hilbert layer.

1.8 Uniqueness of the Perpendicular Foot

The key geometric lemma is

```
theorem perpendicular_foot_unique
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis : ReflectionAxis Geo)
  (P H K : Geo.Point)
  (hPerpH : PerpendicularToAxis Geo axis H P)
  (hPerpK : PerpendicularToAxis Geo axis K P) :
  H = K
```

The proof is neutral.

Assume $H \neq K$. Because both H and K lie on the reflection axis, carrier transport converts the two perpendicularity witnesses into right angles

$$\angle KHP \quad \text{and} \quad \angle HKP.$$

Hence the nondegenerate triangle HKP would have right angles at both H and K .

Extend KH beyond H . The resulting exterior angle is again right. All right angles are congruent, so this exterior angle is congruent to the remote interior right angle at K . Hilbert's exterior-angle theorem forbids this. Therefore $H = K$.

The argument uses no Euclidean parallel axiom. Its mathematical core is

two distinct perpendicular feet $\implies$ triangle with two right angles $\implies \perp$.

1.9 Uniqueness of Reflection

Once the perpendicular foot is unique, uniqueness of the reflected point follows from the existing segment-construction uniqueness theorem.

```

theorem line_reflection_unique
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis : ReflectionAxis Geo)
  (P P1 P2 : Geo.Point)
  (hRef1 : IsLineReflection Geo axis P P1)
  (hRef2 : IsLineReflection Geo axis P P2) :
  P1 = P2

```

In the off-axis case, both reflections have the same perpendicular foot H . Both points P_1, P_2 lie on the prescribed ray beyond H , and their distances are determined by the same midpoint data. Segment additivity and `hilbert_segment_construction_unique` identify the two endpoints.

Thus

$$\forall P \exists! P' \text{ IsLineReflection}(l, P, P').$$

This is the logical threshold at which reflection can be promoted from a relation to a point transformation.

1.10 The Reflection Operator

The actual point transformation is selected noncomputably from the existence theorem:

```

noncomputable def lineReflect
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis : ReflectionAxis Geo)
  (P : Geo.Point) :
  Geo.Point :=
  Classical.choose
  (line_reflection_exists Geo axis P)

```

Its specification theorem is

```

theorem lineReflect_spec
  ...
  (P : Geo.Point) :
  IsLineReflection Geo axis P
  (lineReflect Geo axis P)

```

The use of `Classical.choose` is a packaging step only. The geometric existence and uniqueness were proved before the function was introduced.

1.11 The First Coxeter Relation

The main result of the module is

```
theorem lineReflect_involutive
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis : ReflectionAxis Geo)
  (P : Geo.Point) :
  lineReflect Geo axis
    (lineReflect Geo axis P) = P
```

The proof has a short conceptual form:

$$\begin{array}{l}
 P \ R_l \ r_l(P) \\
 \Downarrow \text{ symmetry} \\
 r_l(P) \ R_l \ P \\
 r_l(P) \ R_l \ r_l(r_l(P)) \\
 \Downarrow \text{ uniqueness} \\
 P = r_l(r_l(P)).
 \end{array}$$

Consequently the first Coxeter relation has now been derived inside the Hilbert model:

$$\boxed{r_l^2 = 1.}$$

This is stronger architecturally than merely assuming an abstract involutory generator: the generator itself has been reconstructed from incidence, order, congruence, midpoint, and right-angle geometry.

1.12 Permutation Packaging

The reflection is finally packaged as a permutation of the point set:

```
noncomputable def lineReflectEquiv
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis : ReflectionAxis Geo) :
  Equiv Geo.Point Geo.Point
```

Both the forward and inverse maps are `lineReflect`; the two inverse laws are exactly `lineReflect_involutive`.

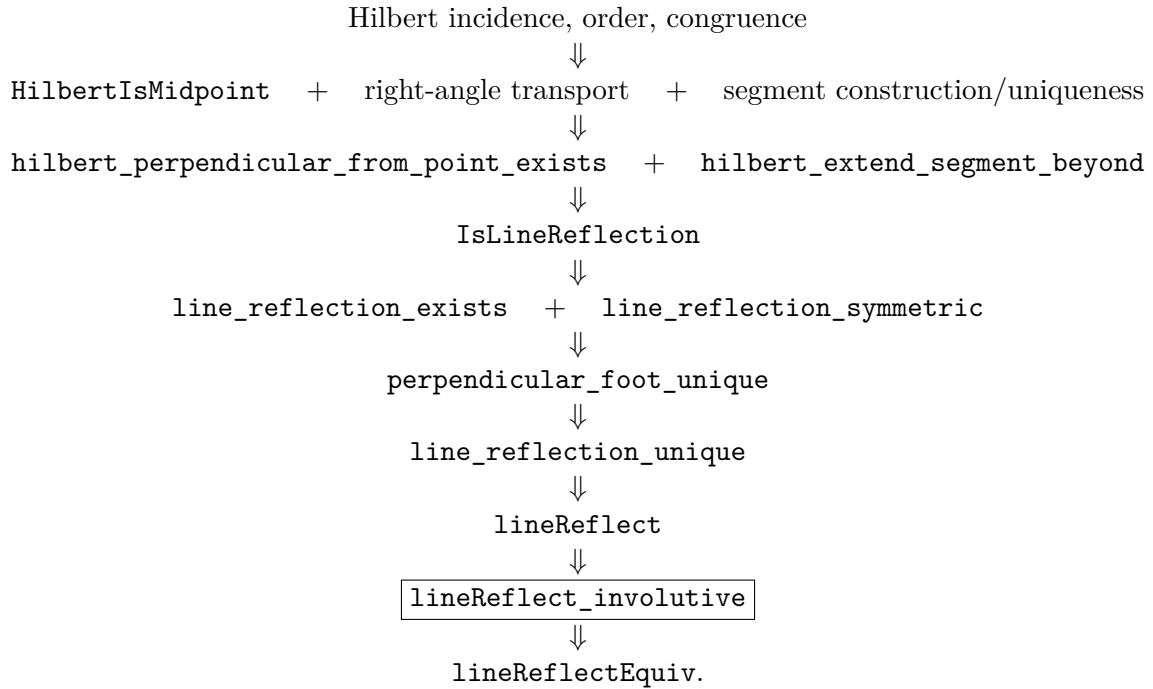
The associated simplification lemmas are

```
lineReflectEquiv_apply
lineReflectEquiv_apply_twice
```

This packaging is the interface needed by the next module, where different reflection generators can be composed into words.

1.13 Dependency Summary

The formal architecture may be summarized as follows:



The only direct Euclid Proposition import required by this module is the Hilbert reconstruction of I.12 for the perpendicular construction. The remaining work belongs to the established Hilbert congruence/order/right angle infrastructure and the local Coxeter helper layer.

1.14 Axiomatic Status

The module introduces no new declared geometric axiom and no **sorry**. The uniqueness of the perpendicular foot is proved by neutral angle theory rather than by uniqueness of parallels.

The transformation **lineReflect** is declared **noncomputable** because it uses **Classical.choose** after existence has been established. This should not be confused with a new geometric axiom.

The exact transitive axiom set should be recorded with **#print axioms** when the Coxeter checkpoint is audited. The present documentation therefore makes the precise local claim: *no new axiom is introduced by Reflection.lean*.

1.15 Architectural Significance

The development establishes a new path in CGJteamLab:

Hilbert geometry $\longrightarrow$ geometrically defined reflection $\longrightarrow$ point permutation $\longrightarrow$ group word.

This is distinct from two earlier proof mechanisms:

- a Pappus-style relabelling argument, where the same points are read in a different order;
- direct Hilbert congruence proofs, where no transformation object is constructed.

Here the transformation is a first-class object derived from the Hilbert geometry. This is the feature required before Coxeter words and pairwise-generator relations can be studied formally.

1.16 Next Checkpoint

No further material should be added to `Reflection.lean` merely to anticipate later needs.

The next experimental module is

```
CGJteamLab/Coxeter/ReflectionWords.lean
```

with the initial task of composing two generators

$$r_l r_m.$$

Only after this composition layer is stable should the project attempt the second Coxeter relation

$$(r_l r_m)^{p_{lm}} = 1$$

for intersecting axes, and the infinite-period case associated with parallel axes.

1.17 Conclusion

The first Coxeter generator has been reconstructed rather than postulated.

Starting from Hilbert incidence, order, congruence, midpoint, and right-angle geometry, the development proves existence and uniqueness of line reflection, promotes the resulting relation to a permutation of the point set, and establishes

$$\boxed{r_l^2 = 1}.$$

This provides the first formal bridge from the synthetic Hilbert language of CGJteamLab to a generators-and-relations language of geometric transformations.

Chapter 2

Words in Reflection Generators

2.1 Purpose of the Module

The module

```
CGJteamLab/Coxeter/ReflectionWords.lean
```

is the first layer in which the geometric reflections constructed in `Reflection.lean` are treated as generators of words.

The previous chapter produced, for every reflection axis l , an equivalence

$$r_l : \text{Point} \simeq \text{Point}$$

satisfying

$$r_l^2 = 1.$$

The present module adds no new geometry. Its role is algebraic packaging: it composes reflection generators, iterates their products, and states the pairwise Coxeter relation in a form that later geometric modules can prove.

Thus the formal transition is

geometric reflection $\longrightarrow$ permutation $\longrightarrow$ word in two generators.

2.2 Product of Two Reflection Generators

For two axes `axis1` and `axis2`, the basic product is

```
noncomputable def reflectionProduct
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis1 axis2 : ReflectionAxis Geo) :
  Equiv Geo.Point Geo.Point :=
  (lineReflectEquiv Geo axis1).trans
  (lineReflectEquiv Geo axis2)
```

The order convention is important. On points,

$$\text{reflectionProduct}(r_1, r_2)(P) = r_2(r_1(P)).$$

This is recorded by the simplification theorem

```
@[simp]
theorem reflectionProduct_apply
...
(P : Geo.Point) :
reflectionProduct Geo axis1 axis2 P =
  lineReflect Geo axis2
    (lineReflect Geo axis1 P)
```

Hence `axis1` acts first and `axis2` second.

This convention is fixed once and used throughout the Coxeter layer. It is therefore possible to read expressions such as

$$r_1 r_2, \quad (r_1 r_2)^2, \quad (r_1 r_2)^p$$

directly in the Lean source without repeatedly reopening the underlying definition.

2.3 The Coincident-Axis Check

The first sanity check is the case in which the two axes coincide.

```
@[simp]
theorem reflectionProduct_self_apply
...
(axis : ReflectionAxis Geo)
(P : Geo.Point) :
reflectionProduct Geo axis axis P = P
```

This theorem is an immediate consequence of `lineReflect_involutive`. It confirms that the algebraic product layer is synchronized with the previously proved geometric relation

$$r_l^2 = 1.$$

The result is small, but architecturally important: the word layer is not an independent abstract group model. It is built directly on the actual Hilbert reflection transformations.

2.4 Powers of a Reflection Product

The module next introduces iteration of the two-generator product:

```
noncomputable def reflectionProductPow
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis1 axis2 : ReflectionAxis Geo)
  (n : Nat) :
  Equiv Geo.Point Geo.Point := ...
```

The associated normalization theorems are

```
reflectionProductPow_zero
reflectionProductPow_succ
reflectionProductPow_one
```

They provide the formal recursion rules needed later to reduce a numerical Coxeter exponent to an explicit word.

Schematically,

$$(r_1 r_2)^0 = 1,$$

and

$$(r_1 r_2)^{n+1} = (r_1 r_2)^n (r_1 r_2).$$

At this stage no claim is made about the order of $r_1 r_2$. The module only constructs the word. Its period must come from geometry of the pair of axes.

2.5 The Pairwise Coxeter Relation

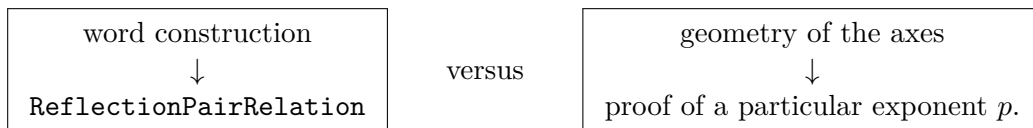
The central definition is

```
def ReflectionPairRelation
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis1 axis2 : ReflectionAxis Geo)
  (p : Nat) : Prop :=
  reflectionProductPow Geo axis1 axis2 p =
  Equiv.refl Geo.Point
```

This is the direct formal counterpart of the pairwise Coxeter relation

$$(r_i r_j)^{p_{ij}} = 1.$$

The definition deliberately contains no geometric hypothesis on the axes. It separates two logically different layers:



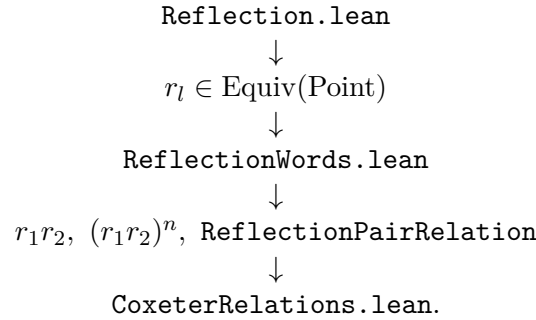
The next chapter supplies the first nontrivial instance: perpendicular axes imply $p = 2$.

2.6 Why the Word Layer is Separate

It would have been possible to define the special relation for perpendicular axes directly in the geometric module. The separate word layer is more useful.

First, it fixes the algebraic language before any particular configuration is studied. Second, the same definitions can be reused for intersecting axes with other finite periods and for parallel axes, where no finite period is expected. Third, the final theorem can be stated in the same form as a Coxeter presentation rather than as an ad hoc pointwise identity.

The architecture is therefore



2.7 Axiomatic Status

The module introduces no new geometric axiom and no new geometric construction. It only packages previously constructed reflections as equivalences and composes them.

The noncomputability is inherited from `lineReflectEquiv`, whose underlying point map ultimately uses `Classical.choose` after geometric existence and uniqueness have already been proved.

The exact transitive axiom set of later final theorems should be recorded separately by `#print axioms`. The local statement for this module is simply that no new foundational assumption is added by the word layer.

2.8 Conclusion

The module completes the passage from a single reflection generator to the language required for pairwise Coxeter relations:

$$\boxed{r_l^2 = 1 \implies r_1 r_2 \implies (r_1 r_2)^p \implies \text{ReflectionPairRelation.}}$$

What remains is no longer to define the word, but to derive its period from the geometry of the reflecting axes.

Chapter 3

Relations Between Two Reflection Generators

3.1 Purpose of the Chapter

This chapter records the first genuine relation between two distinct geometric reflection generators.

The target is the Coxeter case

$$p_{12} = 2.$$

Geometrically, this is the case in which the two reflecting axes are perpendicular. The final theorem is

$$(r_1 r_2)^2 = 1.$$

The proof is not inserted as an abstract group identity. It is reconstructed from the Hilbert geometry of the two axes and from the previously proved metric behavior of line reflection.

Two Lean modules participate:

```
CGJteamLab/Coxeter/ReflectionIsometry.lean
CGJteamLab/Coxeter/CoxeterRelations.lean
```

The first establishes the metric behavior of a single reflection. The second packages perpendicular axes, proves the conjugation identity, and deduces the Coxeter relation.

3.2 Reflection as a Synthetic Isometry

The decisive theorem of `ReflectionIsometry.lean` is

```
theorem lineReflect_preserves_congruence
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis : ReflectionAxis Geo)
  (P Q : Geo.Point) :
  Geo.Congruent
    P Q
    (lineReflect Geo axis P)
```

```
(lineReflect Geo axis Q)
```

Thus a line reflection preserves every segment congruence.

This is derived synthetically, without introducing coordinates or a numerical distance function. The proof separates several geometric configurations according to the perpendicular feet of the two points and uses the already established Hilbert congruence and right-angle theory.

Once segment congruence is available, the relation module extends the isometry package to the structural relations needed by conjugation:

```
lineReflect_preserves_between
lineReflect_preserves_collinear
lineReflect_preserves_noncollinear
lineReflect_preserves_angle
lineReflect_preserves_right_angle
```

The logical chain is

segment congruence $\longrightarrow$ betweenness and collinearity $\longrightarrow$ SSS $\longrightarrow$ angle congruence $\longrightarrow$ right-angle preservation

This is precisely the transformation-theoretic package required to move a reflection configuration through another reflection.

3.3 Perpendicular Reflection Axes

Perpendicularity of the two reflection axes is represented by explicit witness data:

```
structure ReflectionAxesPerpendicular
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis1 axis2 : ReflectionAxis Geo) where
  O : Geo.Point
  U : Geo.Point
  V : Geo.Point
  hO1 : HilbertIncidence.OnLine O axis1.carrier
  hO2 : HilbertIncidence.OnLine O axis2.carrier
  hU1 : HilbertIncidence.OnLine U axis1.carrier
  hV2 : HilbertIncidence.OnLine V axis2.carrier
  hOU : Not (O = U)
  hOV : Not (O = V)
  hNonCol : Not (PrimCollinear Geo U O V)
  hRight : HilbertRightAngle Geo U O V
```

The certificate makes all hidden diagrammatic information explicit. The axes meet at O ; the points U and V determine nondegenerate directions on the two carriers; and $\angle UOV$ is right.

Perpendicularity is then shown to be symmetric by `reflectionAxesPerpendicular_symm`.

3.4 Setwise Preservation of the Perpendicular Axis

The first geometric consequence is that reflection in one axis preserves the other perpendicular axis setwise.

The key theorems are

```
perpendicular_axes_second_off_first
perpendicular_axes_second_point_perpendicular_first
lineReflect_preserves_perpendicular_axis_second
lineReflect_preserves_perpendicular_axis_first
```

Their geometric content is simple but formally important.

Let $P \in l_2$ with $P \neq O$. Then $P \notin l_1$, otherwise the two carriers would share two distinct points and therefore coincide, contradicting the nondegenerate right-angle configuration.

Moreover OP is perpendicular to l_1 . Therefore O is the perpendicular foot used in the reflection of P across l_1 . The midpoint condition gives

$$P - O = r_1(P),$$

so $r_1(P)$ remains on the carrier l_2 .

Hence

$$r_1(l_2) = l_2, \quad r_2(l_1) = l_1.$$

This setwise invariance is necessary, but by itself it does not yet prove that the two reflections commute.

3.5 Transport of Midpoints and Perpendiculars

The conjugation argument requires more than preservation of the axis. A complete reflection witness consists of a perpendicular foot together with a midpoint condition.

The theorem

```
theorem lineReflect_preserves_midpoint
...
(hMid : HilbertIsMidpoint Geo M A B) :
HilbertIsMidpoint
  Geo
    (lineReflect Geo axis M)
    (lineReflect Geo axis A)
    (lineReflect Geo axis B)
```

uses preservation of strict betweenness and of segment congruence.

The second transport theorem is

```

theorem perpendicular_axes_transport_second_perpendicular
...
(hPerp :
  PerpendicularToAxis Geo axis2 H P) :
PerpendicularToAxis
  Geo
  axis2
  (lineReflect Geo axis1 H)
  (lineReflect Geo axis1 P)

```

Here the invariant carrier l_2 remains the target axis, while the foot, the off-axis point, and the right-angle witness are all transported through r_1 .

This is the technical core of the conjugation proof.

3.6 Conjugation

The central pointwise theorem is

```

theorem perpendicular_axes_conjugation_pointwise
...
(P : Geo.Point) :
lineReflect Geo axis1
  (lineReflect Geo axis2
    (lineReflect Geo axis1 P)) =
lineReflect Geo axis2 P

```

In mathematical notation,

$$r_1 r_2 r_1 = r_2.$$

The proof splits according to whether P lies on l_2 .

If $P \in l_2$, then $r_1(P) \in l_2$, so r_2 fixes both relevant points and the conclusion reduces to $r_1^2 = 1$.

Assume now $P \notin l_2$. Reflect $r_1(P)$ in l_2 , obtaining a perpendicular foot H and midpoint configuration

$$r_1(P) - H - r_2(r_1(P)).$$

Apply r_1 to the whole configuration. Preservation of the perpendicular data and of the midpoint gives

$$P - r_1(H) - r_1 r_2 r_1(P),$$

with $r_1(H) \in l_2$ and the required right-angle condition.

Therefore $r_1 r_2 r_1(P)$ satisfies the defining relation `IsLineReflection` for the original point P and the axis l_2 .

But the reflected point in a fixed axis is unique. Hence

$$r_1 r_2 r_1(P) = r_2(P).$$

This is the exact point where the earlier theorem `line_reflection_unique` re-enters the Coxeter development.

3.7 Commutation

Applying the conjugation identity to $r_1(P)$ and using $r_1^2 = 1$ gives

```
theorem perpendicular_axes_reflections_commute_pointwise
...
(P : Geo.Point) :
lineReflect Geo axis1
  (lineReflect Geo axis2 P) =
lineReflect Geo axis2
  (lineReflect Geo axis1 P)
```

Thus

$$r_1 r_2 = r_2 r_1$$

pointwise.

The equality is also packaged at the permutation level:

```
theorem perpendicular_axes_reflectionProduct_commute
...
reflectionProduct Geo axis1 axis2 =
reflectionProduct Geo axis2 axis1
```

This is the permutation-level form of the classical fact that reflections in perpendicular lines commute.

3.8 The Square of the Product

From the conjugation identity one obtains directly

```
theorem perpendicular_axes_reflectionProduct_square_apply
...
(P : Geo.Point) :
reflectionProduct Geo axis1 axis2
  (reflectionProduct Geo axis1 axis2 P) =
P
```

The pointwise result is then lifted to an equality of equivalences:

```
theorem perpendicular_axes_reflectionProductPow_two
...
reflectionProductPow Geo axis1 axis2 2 =
Equiv.refl Geo.Point
```

The proof of this wrapper is intentionally thin. All substantive geometry has already been completed before this stage.

3.9 The Coxeter Relation $p_{12} = 2$

The final theorem of the present checkpoint is

```

theorem perpendicular_axes_coxeter_relation_two
  [HilbertIncidence Geo]
  [HilbertCongruence Geo]
  (axis1 axis2 : ReflectionAxis Geo)
  (hAxes :
    ReflectionAxesPerpendicular Geo axis1 axis2) :
  ReflectionPairRelation
    Geo
    axis1 axis2
    2

```

Unfolding `ReflectionPairRelation`, this is exactly

$$\boxed{(r_1 r_2)^2 = 1}.$$

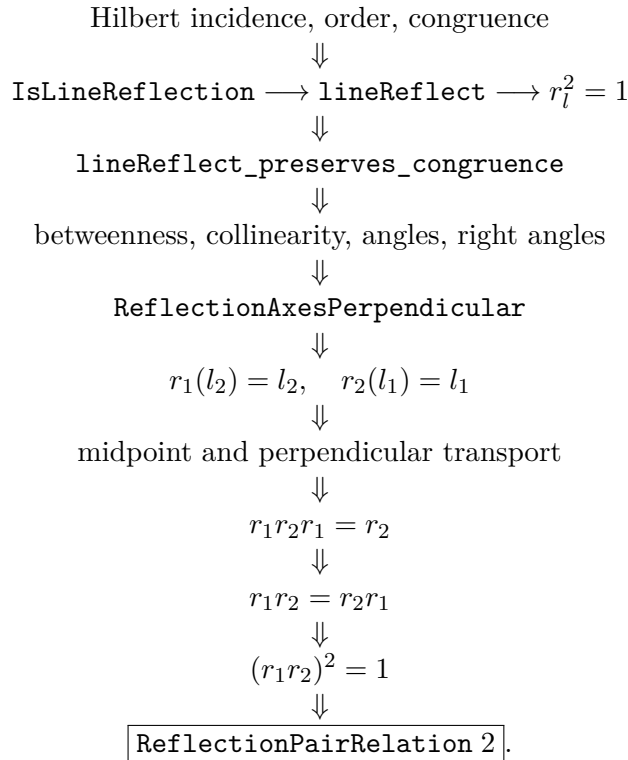
Thus the first nontrivial pairwise Coxeter exponent has been derived from the geometry of the mirrors:

$$\boxed{l_1 \perp l_2 \implies p_{12} = 2.}$$

The theorem is stronger architecturally than an abstract presentation assumption. The generators themselves were reconstructed from Hilbert geometry, and the exponent 2 is derived from the geometric configuration of their axes.

3.10 Dependency Architecture

The completed proof may be summarized by the following DAG:



This is the first complete passage in the project from synthetic geometry to a nontrivial relation among several Coxeter generators.

3.11 Axiomatic Status

The two modules documented here introduce no new declared geometric axiom and no `sorry`.

The public relation theorem is stated with only `HilbertIncidence` and `HilbertCongruence` instances. The actual transitive axiom footprint of the final theorem should still be recorded explicitly with

```
#print axioms Geometry.perpendicular_axes_coxeter_relation_two
```

when the Coxeter layer is audited.

The present documentation therefore makes the local claim that the $p = 2$ relation has been proved inside the already established Hilbert infrastructure without adding a new Coxeter-specific geometric axiom.

3.12 Conclusion

The completed chapter establishes the chain

$$\boxed{\text{perpendicular mirrors} \implies r_1 r_2 r_1 = r_2 \implies r_1 r_2 = r_2 r_1 \implies (r_1 r_2)^2 = 1.}$$

In the language prepared by `ReflectionWords.lean`, this becomes

ReflectionPairRelation 2.

The next mathematical problem is no longer the perpendicular case. It is to determine how the angle between two intersecting mirrors controls the general finite exponent p_{ij} .

Appendix A

Source Map: Coxeter–Moser

The supplement currently uses the following parts of Coxeter–Moser as conceptual control.

A.1 Section 1.1: Generators and Relations

This section supplies the abstract language of generators, words, relations, and presentations.

A.2 Section 4.1: Cyclic and Dihedral Groups

This section provides the elementary geometric interpretation of dihedral groups in terms of rotations and reflections.

A.3 Section 4.3: Groups Generated by Reflections

This section connects words in reflection generators with paths through fundamental regions and explains how local geometric circuits produce defining relations.

A.4 Sections 9.1–9.3: Groups Generated by Reflections

These sections provide the general Coxeter-type presentation

$$r_i^2 = 1, \quad (r_i r_j)^{p_{ij}} = 1,$$

the graphical notation, and the realization of generators by real affine reflections.

Appendix B

Lean Module Map

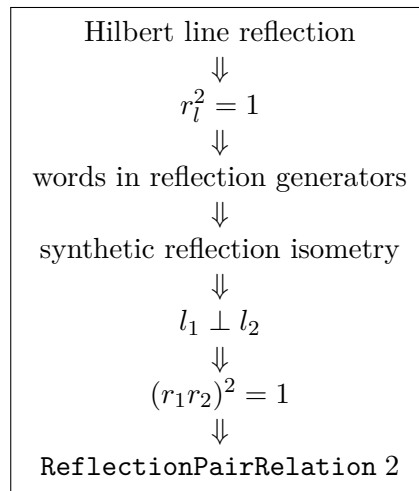
The current module architecture is

```
CGJteamLab/Coxeter/  
  Reflection.lean  
  ReflectionWords.lean  
  ReflectionIsometry.lean  
  CoxeterRelations.lean  
  EuclidBenchmarks.lean      -- planned
```

The completed Coxeter layer currently consists of `Reflection.lean`, `ReflectionWords.lean`, `ReflectionIsometry.lean`, and `CoxeterRelations.lean`.

Checkpoint

Current completed milestone:



The next mathematical target is the general finite-period case for intersecting axes: derive the exponent p_{ij} from the geometry of the angle between the two reflecting carriers.